

Permainan Memasang Ubin

Barchin dan Charos sedang bermain pada *grid* yang terdiri dari $2N \times 2M$ sel persegi satuan. Baris-baris dinomori 0 hingga $2N - 1$ dari atas ke bawah, dan kolom-kolom dinomori 0 hingga $2M - 1$ dari kiri ke kanan. Untuk $0 \leq i < 2N$ dan $0 \leq j < 2M$, kita sebut sel pada baris i dan kolom j sebagai (i, j) .

Barchin memberikan $N \cdot M$ **balok** kepada Charos satu per satu. Setiap balok berupa persegi 2×2 yang terdiri dari empat **ubin** 1×1 . Barchin telah mewarnai setiap ubin dari balok tersebut dengan warna hitam atau putih, dengan menjamin bahwa **setidaknya satu ubin berwarna putih**.

Charos harus meletakkan setiap balok pada *grid* **langsung setelah ia menerimanya**, tanpa mengetahui balok-balok yang akan ia terima setelahnya. Balok-balok tidak dapat diputar. Setiap balok harus diletakkan sepenuhnya di dalam *grid* dan menutupi tepat empat sel *grid*. Selain itu, ubin kiri atas setiap balok harus menutupi sel yang koordinat baris dan kolomnya genap. Setiap sel pada *grid* dapat ditutupi paling banyak satu balok.

Barchin memenangkan permainan jika setelah suatu balok ditempatkan, terdapat wilayah persegi sel-sel berukuran 2×2 yang tertutupi oleh empat ubin hitam. Secara formal, jika sel (a, b) , $(a + 1, b)$, $(a, b + 1)$, $(a + 1, b + 1)$ semuanya tertutupi oleh ubin hitam untuk suatu $0 \leq a < 2N - 1$ dan $0 \leq b < 2M - 1$, Barchin menang. Nilai a dan b tidak harus genap.

Charos menang jika ia berhasil meletakkan semua $N \cdot M$ balok dan Barchin tidak pernah menang. Perhatikan bahwa meletakkan $N \cdot M$ balok akan menutupi seluruh *grid*.

Tugas Anda adalah menerapkan suatu strategi agar Charos menang. Dapat dibuktikan bahwa, di bawah batasan yang diberikan, Charos selalu dapat menempatkan balok-balok sedemikian rupa sehingga kemenangannya terjamin, terlepas dari warna balok yang akan diterimanya.

Detail Implementasi

Anda harus mengimplementasikan dua prosedur:

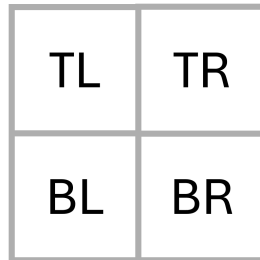
```
void init(int N, int M)
```

- N : setengah dari banyaknya baris pada *grid*.
- M : setengah dari banyaknya kolom pada *grid*.

- Prosedur ini dipanggil tepat satu kali untuk setiap kasus uji di awal penjalanan program Anda.

```
std::pair<int, int> receive_block(int TL, int TR, int BL, int BR)
```

- TL, TR, BL, BR : warna ubin kiri atas, kanan atas, kiri bawah, dan kanan bawah dari balok saat ini secara berturut-turut, seperti ditunjukkan pada gambar di bawah. Nilai masing-masing parameter adalah 0 (putih) atau 1 (hitam).
- Prosedur ini dipanggil tepat $N \cdot M$ kali untuk setiap kasus uji setelah pemanggilan awal `init`.



Prosedur ini harus mengembalikan sepasang bilangan bulat (i, j) , dengan i menyatakan koordinat baris dan j menyatakan koordinat kolom dari sel tempat ubin kiri atas balok ini diletakkan. Nilai i dan j harus genap, dan wilayah 2×2 yang dicakup balok tersebut tidak boleh tumpang tindih dengan balok-balok yang telah ditempatkan sebelumnya.

Jika `receive_block` mengembalikan suatu pasangan yang tidak memenuhi persyaratan ini, atau jika setelah menempatkan suatu balok, suatu wilayah persegi sel 2×2 sepenuhnya tertutup ubin hitam, *grader* akan langsung menghentikan program Anda dan hasil kasus uji tersebut adalah `Output isn't correct`.

Perilaku *grader* **tidak adaptif**. Ini berarti urutan balok-balok yang diberikan Barchin kepada Charos telah ditetapkan sebelum `init` dipanggil.

Batasan

Untuk setiap balok, sebut S sebagai banyaknya ubin hitam di antara keempat ubinnya. Dengan kata lain, $S = TL + TR + BL + BR$.

- $1 \leq N, M \leq 100$
- $0 \leq S \leq 3$ untuk setiap balok.

Subsoal

Subsoal	Nilai	Batasan Tambahan
1	6	$S = 1$ untuk setiap balok, dan $N = 2$.
2	16	$S = 3$ untuk setiap balok. $N = M$, N genap, dan masing-masing kemungkinan konfigurasi pewarnaan balok muncul tepat $\frac{N^2}{4}$ kali.
3	10	$S = 1$ untuk setiap balok.
4	29	$S \leq 2$ untuk setiap balok.
5	39	Tidak ada batasan tambahan.

Contoh

Perhatikan suatu permainan dengan $N = 1$ dan $M = 2$, sehingga *grid*-nya memiliki 2 baris dan 4 kolom. *Grader* pertama-tama memanggil:

```
init(1, 2)
```

Awalnya, semua sel kosong. *Grid*-nya tampak sebagai berikut:

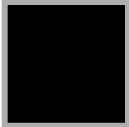
	0	1	2	3
0				
1				

Ada $N \cdot M = 2$ balok yang harus ditempatkan. Misalkan Barchin memberikan sebuah balok dengan tiga ubin hitam dan satu ubin putih di sudut kanan atas. *Grader* memanggil:

```
receive_block(1, 0, 1, 1)
```

Charos memutuskan untuk menempatkan balok ini di bagian kiri *grid* dengan mengembalikan $(0, 0)$.

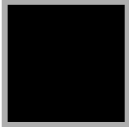
Grid-nya kini tampak sebagai berikut:

	0	1	2	3
0				
1				

Barchin kemudian memberikan balok lain dengan tiga ubin hitam dan satu ubin putih di sudut kiri atas:

```
receive_block(0, 1, 1, 1)
```

Satu-satunya sel yang tersisa dengan baris genap dan kolom genap yang dapat menjadi sudut kiri atas balok 2×2 adalah $(0, 2)$, sehingga Charos mengembalikan $(0, 2)$. *Grid* akhir adalah sebagai berikut:

	0	1	2	3
0				
1				

Tidak ada wilayah persegi 2×2 yang sepenuhnya tertutup oleh ubin hitam, sehingga Charos berhasil menempatkan semua balok dan Barchin tidak pernah menang. Charos pun memenangkan permainan.

Contoh Grader

Format masukan:

```
N M
TL[0] TR[0] BL[0] BR[0]
TL[1] TR[1] BL[1] BR[1]
...
TL[NM-1] TR[NM-1] BL[NM-1] BR[NM-1]
```

Format keluaran:

```
R[0] C[0]  
R[1] C[1]  
...  
R[NM-1] C[NM-1]
```

Di sini, $R[k]$ dan $C[k]$ adalah pasangan bilangan bulat yang dikembalikan oleh pemanggilan ke- k dari `receive_block`.